

1 DDL — создание и изменение структуры

CREATE TABLE · ALTER TABLE · DROP TABLE · первичный и внешний ключи (PK → FK)

В русской локали Access в выражениях часто используется ; вместо ,

CREATE TABLE создание таблицы

Создаёт новую таблицу: перечисляются поля, их типы данных и ключ.

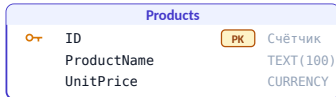
СИНТАКСИС

```
CREATE TABLE таблица (  
    поле1 тип [PRIMARY KEY],  
    поле2 тип, ...);
```

ПРИМЕР — ОДНА ТАБЛИЦА

```
CREATE TABLE Products (  
    ID AUTOINCREMENT PRIMARY KEY,  
    ProductName TEXT(100),  
    UnitPrice CURRENCY);
```

Результат: пустая таблица **Products** с ключевым полем **ID**, текстовым и денежным полями.



поля и типы
в скобках через запятую;
записей пока 0 —
структура готова

ALTER TABLE изменение структуры

Добавляет новое поле или меняет тип существующего — без пересоздания таблицы.

СИНТАКСИС

```
ALTER TABLE таблица ADD COLUMN имя тип;  
ALTER TABLE таблица ALTER COLUMN имя тип;
```

ПРИМЕР — ДОБАВИТЬ ПОЛЕ

```
ALTER TABLE Products  
    ADD COLUMN CategoryGroup TEXT(40);
```

ПРИМЕР — РАСШИРИТЬ ПОЛЕ ДО 255 СИМВОЛОВ

```
ALTER TABLE Products  
    ALTER COLUMN CategoryGroup TEXT(255);
```

Результат: у всех существующих записей новое поле равно **Null**, пока его не заполнят.



CREATE TABLE две связанные таблицы — PK → FK

Сначала создаётся таблица-справочник, затем таблица со ссылкой на её ключ: поле **CategoryID** в Products объявляется типом **LONG** и связывается ограничением FOREIGN KEY.

ПРИМЕР — СПРАВОЧНИК И ТАБЛИЦА ТОВАРОВ СО СВЯЗЬЮ

```
CREATE TABLE Categories (  
    CategoryID AUTOINCREMENT PRIMARY KEY,  
    CategoryName TEXT(60) NOT NULL);
```

```
CREATE TABLE Products (  
    ProductID AUTOINCREMENT PRIMARY KEY,  
    ProductName TEXT(100) NOT NULL,  
    UnitPrice CURRENCY,  
    CategoryID LONG,  
    CONSTRAINT FK_Products_Categories  
        FOREIGN KEY (CategoryID)  
        REFERENCES Categories (CategoryID));
```

Результат: вставить в **Products.CategoryID** значение, которого нет в **Categories**, Access не даст — связь 1:∞ под защитой.



PK и FK — роль ключей

PK	уникальный идентификатор записи своей таблицы
FK	ссылка на запись (PK) другой таблицы
NOT NULL	поле обязательно для заполнения
UNIQUE	значения в столбце не повторяются



NOT NULL · UNIQUE ограничения столбца

NOT NULL запрещает пустые значения, UNIQUE — повторы в столбце.

ПРИМЕР — ОБА ОГРАНИЧЕНИЯ

```
CREATE TABLE Categories (  
    CategoryID AUTOINCREMENT PRIMARY KEY,  
    CategoryName TEXT(60) NOT NULL);  
ALTER TABLE Categories  
    ADD CONSTRAINT Uq_Name UNIQUE (CategoryName);
```

Результат: пустое имя категории и дубликат Access отклонит при вводе.

"Электроника"	✓ принято
"Электроника" — повтор	✗ UNIQUE
Null — пусто	✗ NOT NULL

DROP TABLE удаление таблицы

Удаляет таблицу целиком — и структуру, и все данные. Действие необратимо.

СИНТАКСИС

```
DROP TABLE таблица;
```

ПРИМЕР

```
DROP TABLE Products;
```

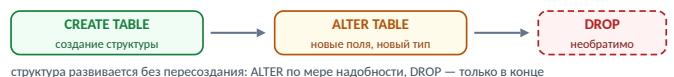
Результат: таблица **Products** исчезает из схемы данных вместе со всеми записями.



Практика: если на таблицу ссылается связь FK, сначала снимите связь — и только затем выполняйте DROP TABLE.

Жизненный цикл таблицы

Три DDL-оператора покрывают всю жизнь таблицы: создать — доработать — удалить.



2 DML — добавление, изменение, удаление

INSERT INTO · UPDATE · DELETE — для каждого оператора сцена «данные до → действие → данные после»

Перед UPDATE и DELETE выполните SELECT с тем же WHERE

INSERT INTO одна запись

Добавляет в таблицу одну новую строку: список полей затем список значений.

СИНТАКСИС

```
INSERT INTO таблица (поле1, поле2)
VALUES (значение1, значение2);
```

ПРИМЕР — ОДНО ЗНАЧЕНИЕ

```
INSERT INTO Products (ProductName, UnitPrice)
VALUES ("Стол", 25000);
```

ProductName	UnitPrice		ProductName	UnitPrice
Стул	4500	→ INSERT	Стул	4500
Лампа	1200		Лампа	1200
			Стол	25000

Результат: строка «Стол» добавлена в конец таблицы; ID ей присвоит счётчик.

UPDATE изменить одно поле

Меняет значения полей у записей, отобранных условием WHERE. Без WHERE изменятся все строки!

СИНТАКСИС

```
UPDATE таблица SET поле = значение
WHERE условие;
```

ПРИМЕР — ОДНО ЗНАЧЕНИЕ

```
UPDATE Products SET UnitPrice = 30000
WHERE ID = 1;
```

ID	UnitPrice		ID	UnitPrice
1	25000	→ UPDATE	1	30000
2	1200		2	1200

Результат: цена записи ID = 1 изменена на 30000; остальные не тронуты.

DELETE удаление записей

Удаляет целиком строки, отобранные условием WHERE. Без WHERE удалятся все данные таблицы!

СИНТАКСИС

```
DELETE FROM таблица WHERE условие;
```

ПРИМЕР — ОДНА ИЛИ НЕСКОЛЬКО ЗАПИСЕЙ

```
DELETE FROM Products WHERE ID = 1;
```

ID	ProductName		ID	ProductName
1	Стол	→ DELETE	2	Лампа
2	Лампа			

Результат: строка ID = 1 удалена целиком; поля удалять по отдельности нельзя.

INSERT INTO ... SELECT несколько записей

Копирует в таблицу сразу набор строк, отобранных запросом SELECT.

СИНТАКСИС

```
INSERT INTO таблица_приёмник (список полей)
SELECT ... FROM ... [WHERE ...];
```

ПРИМЕР — НЕСКОЛЬКО ЗНАЧЕНИЙ (АРХИВАЦИЯ)

```
INSERT INTO Products_Archive
SELECT * FROM Products
WHERE UnitPrice < 5000;
```

Products	цена		Products_Archive
Стул	4500	→ копия	
Лампа	1200		Лампа · 1200
Стол	25000		

Результат: все строки с ценой ниже 5000 попали в архив одной командой.

UPDATE несколько полей и выражения

В SET через запятую можно обновить сразу несколько полей — в том числе выражениями над другими полями.

ПРИМЕР — НЕСКОЛЬКО ЗНАЧЕНИЙ СРАЗУ

```
UPDATE Products
SET CategoryGroup = "Электроника",
    UnitPriceNum = CCur(Val(UnitPrice))
WHERE Category IN ("Телефоны", "Компьютеры");
```

до	Group / Num		после	Group / Num
Телефон	Null · Null	→ 2 поля	Телефон	Электр. · 12000
Стол	Мебель · 25000		Стол	Мебель · 25000

Результат: у отобранных категорий заполнены сразу два поля; текст цены превращён в число.

Порядок безопасного изменения

UPDATE и DELETE нельзя отменить в Access — сначала посмотрите, что попадёт под условие.

СХЕМА БЕЗОПАСНОЙ ПРОВЕРКИ



Пример: для `DELETE FROM Products WHERE ID = 1;` сначала выполните `SELECT * FROM Products WHERE ID = 1;`

3 SELECT — выборка данных

выбор полей · DISTINCT · псевдонимы AS · сортировка ORDER BY · первые записи TOP

Фильтр строк делает WHERE — см. блок 4

SELECT * все поля

Звёздочка выбирает все столбцы таблицы — быстрый просмотр целиком.

СИНТАКСИС

```
SELECT * FROM таблица;
```

ПРИМЕР

```
SELECT * FROM Products;
```

Результат: все строки и все столбцы **Products** без фильтрации и сортировки.

Products		
ID	ProductName	UnitPrice
1	Стол	25000
2	Лампа	1200

* = все столбцы
все строки таблицы

SELECT отдельные поля

Перечисление столбцов через запятую оставляет только нужные поля — запрос быстрее и понятнее.

СИНТАКСИС

```
SELECT поле1, поле2 FROM таблица;
```

ПРИМЕР

```
SELECT ProductName, UnitPrice FROM Products;
```

Результат: два столбца **ProductName** и **UnitPrice**; столбец ID в вывод не попадает.

ID	ProductName	UnitPrice

выбраны 2 столбца
ID остаётся в таблице,
но не выводится

DISTINCT без повторов

Убирает дубликаты из результата: каждая комбинация значений выводится один раз.

СИНТАКСИС

```
SELECT DISTINCT поле FROM таблица;
```

ПРИМЕР

```
SELECT DISTINCT Category FROM Products;
```

Category	Category
Электроника	Электроника
Электроника	Авто
Авто	

склопывание



одна строка
Авто — уникальное,
проходит как есть

Результат: список категорий без повторов — удобно для справочников и выпадающих списков.

AS псевдоним столбца

Задаёт заголовок столбца в результате — своё имя для поля или выражения.

СИНТАКСИС

```
SELECT поле AS псевдоним FROM таблица;
```

ПРИМЕР

```
SELECT ProductName AS Товар FROM Products;
```



заголовок в
результате

Результат: данные те же, но столбец подписан **«Товар»** — удобно для выгрузок и отчётов.

ORDER BY сортировка

Сортирует результат: ASC — по возрастанию (по умолчанию), DESC — по убыванию.

СИНТАКСИС

```
ORDER BY поле [ASC | DESC];
```

ПРИМЕР — ПО УБЫВАНИЮ ЦЕНЫ

```
SELECT ProductName, UnitPrice  
FROM Products ORDER BY UnitPrice DESC;
```

Стол	25000
Стул	4500
Лампа	1200

DESC — по убыванию
ASC — по возрастанию (по умолчанию)
сортировка выполняется после фильтра WHERE

TOP первые записи

Возвращает заданное число первых строк — почти всегда вместе с ORDER BY, иначе строки непредсказуемы.

СИНТАКСИС

```
SELECT TOP N поля FROM таблица ORDER BY ...;
```

ПРИМЕР — ТОП-2 ДОРОГИХ ТОВАРА

```
SELECT TOP 2 ProductName, UnitPrice  
FROM Products ORDER BY UnitPrice DESC;
```

1	Стол	25000
2	Стул	4500
3	Лампа	1200

TOP 2 — отрезает
первые 2 строки
порядок задаёт ORDER BY

Порядок частей запроса SELECT



Access выполняет части запроса именно в этом порядке: FROM находит таблицу → WHERE отбирает строки → GROUP BY группирует → HAVING фильтрует группы → ORDER BY сортирует → SELECT возвращает поля.

4 Условия отбора — WHERE

= · IN · BETWEEN · LIKE · IS NULL / IS NOT NULL · AND · OR · NOT — каждый оператор фильтрует строки по-своему

Текст — в кавычках "так"; даты — между решётками #2024-03-01#

= точное равенство

Оставляет записи, у которых поле равно одному конкретному значению.

ПРИМЕР

```
WHERE Category = "Электроника";
```

Category проходит?

Электроника

✓

Авто

✗



фильтр-воронка

сквозь неё проходят только записи, у которых поле точно равно значению

IN список значений

Равенство любому из перечисленных значений — короткая замена цепочке OR.

ПРИМЕР

```
WHERE Category IN ("Телефоны", "Компьютеры");
```

Category проходит?

Телефоны

✓ в списке

Компьютеры

✓ в списке

Авто

✗ нет



список-«ловушка»

значение сравнивается с каждым элементом списка — короткая запись OR

BETWEEN диапазон

Значение внутри отрезка — обе границы включаются в отбор.

ПРИМЕР

```
WHERE UnitPrice BETWEEN 1000 AND 50000;
```

UnitPrice проходит?

25000

✓ внутри

750

✗ меньше

границы входят



IS NULL пустое значение

Находит записи, где поле вообще не заполнено (Null). Сравнение = Null не работает — только IS NULL.

ПРИМЕР

```
WHERE CategoryGroup IS NULL;
```

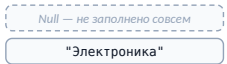
CategoryGroup проходит?

Null

✓ пусто

"Электроника"

✗



✓ IS NULL

= Null не работает: сравнение с неизвестным всегда даёт «неизвестно»

IS NOT NULL непустое значение

Обратно: отбирает только заполненные поля — полезно перед отчётами и выгрузками.

ПРИМЕР

```
WHERE CategoryGroup IS NOT NULL;
```

CategoryGroup проходит?

"Электроника"

✓ заполнено

Null

✗ пропущено



отчёты и выгрузки видят только реальные заполненные данные

AND оба условия

Запись проходит, только если выполняются все условия одновременно.

ПРИМЕР

```
WHERE Category = "Авто" AND UnitPrice < 10000;
```

запись условия

Авто · 150000

✗ цена

Авто · 8500

✓ и то и то



пересечение кружков

запись выживает, только если верно и первое, и второе условие сразу

OR хотя бы одно

Запись проходит, если выполняется хотя бы одно из условий.

ПРИМЕР

```
WHERE Category = "Авто" OR Category = "Дом и мебель";
```

Category условия

Авто

✓ первое

Дом и мебель

✓ второе

Электроника

✗ ни одно



объединение кружков

запись проходит, если верно хотя бы одно из условий — весь силуэт целиком

NOT отрицание

Отбирает записи, не удовлетворяющие условию — «всё, кроме...».

ПРИМЕР

```
WHERE NOT Category = "Электроника";
```

Category NOT =

Электроника

✗ исключена

Авто

✓ проходит



всё, кроме кружка A

зелёное кольцо — записи, которые условие пропускает; внутри — отброшенные

Приоритет: NOT → AND → OR

Сначала выполняется NOT, затем AND, затем OR. Скобки меняют порядок и делают условие читаемым.

ПРИМЕР СО СКОБКАМИ

```
WHERE Category = "Авто"  
OR (Category = "Дом и мебель" AND UnitPrice < 10000);
```

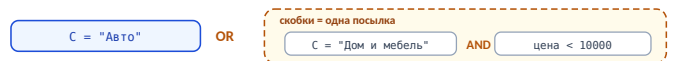
без скобок ошибка смысла

Дом и мебель · 25000

✗ пройдёт?

со скобками

✓ отсеется



скобки = одна посылка

C = "Дом и мебель" AND цена < 10000

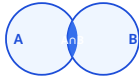
INNER JOIN только совпадения

В результат попадают только записи, у которых нашлась пара в другой таблице.

ПРИМЕР

```
SELECT Products.ProductName,
       Categories.CategoryName
FROM Products INNER JOIN Categories
ON Products.CategoryID = Categories.ID;
```

Результат: товары, у которых заполнена категория; товар без пары пропадает.



только пересечение
записи с парой с обеих сторон
A — Products, B — Categories

LEFT JOIN все слева + совпадения

Все записи левой таблицы; для тех, у кого нет пары, поля правой заполняются Null.

ПРИМЕР

```
SELECT Products.ProductName,
       Categories.CategoryName
FROM Products LEFT JOIN Categories
ON Products.CategoryID = Categories.ID;
```

Результат: весь список товаров, даже без категории (категория = Null).



весь круг A
+ совпадения из B
без пары → Null

RIGHT JOIN все справа + совпадения

Зеркально LEFT: все записи правой таблицы; без пары слева — Null в полях левой.

ПРИМЕР

```
SELECT P.ProductName, C.CategoryName
FROM Products AS P RIGHT JOIN Categories AS C
ON P.CategoryID = C.ID;
```

Результат: все категории, даже пустые — те, где нет ни одного товара.



весь круг B
+ совпадения из A
удобно искать пустые справочники

CROSS JOIN каждая с каждой

Декартово произведение: каждая строка левой таблицы сочетается с каждой строкой правой. В Access — запятая вместо ключевого слова.

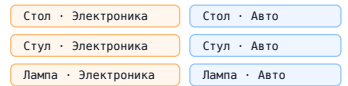
ПРИМЕР

```
SELECT Count(*) FROM Products, Categories;
```

Результат: количество строк = **строки Products × строки Categories** (3 × 2 = 6).



A — 3 строки, B — 2 строки
пересечение не нужно:
берутся все комбинации



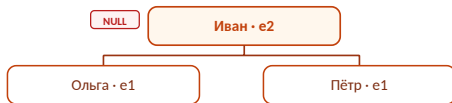
3 × 2 = 6 пар — все сочетания

SELF JOIN таблица с самой собой

Таблица соединяется с собой через два псевдонима — например, сотрудники и их руководители по полю ManagerID.

ПРИМЕР — КТО КОМУ ПОДЧИНЯЕТСЯ

```
SELECT e1.EmployeeName AS Сотрудник,
       e2.EmployeeName AS Руководитель
FROM Employees AS e1 LEFT JOIN Employees AS e2
ON e1.ManagerID = e2.EmployeeID;
```



e1.ManagerID = e2.EmployeeID
LEFT JOIN обязателен: у Иван: руководителя нет (ManagerID одна таблица — две роли чер

Псевдонимы и связь по нескольким полям

Псевдонимы сокращают запись; если таблицы связываются по составному ключу — условия перечисляются через AND.

СОЕДИНЕНИЕ ПО ОДНОМУ ПОЛЮ (ЧЕРЕЗ ПСЕВДОНИМЫ)

```
FROM Products AS P INNER JOIN Categories AS C
ON P.CategoryID = C.ID
```

СОЕДИНЕНИЕ ПО НЕКОЛЬКИМ ПОЛЯМ — УСЛОВИЯ ЧЕРЕЗ AND

```
FROM T1 AS a INNER JOIN T2 AS b
ON a.Key1 = b.Key1 AND a.Key2 = b.Key2
```

Результат: обе части пары ключей должны совпасть одновременно — так соединяются таблицы со сложным ключом.

FULL OUTER JOIN в Access не поддерживается — имитация через UNION

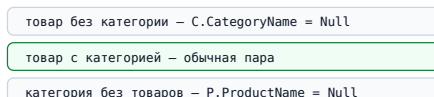
Полное внешнее соединение (все записи обеих таблиц) напрямую недоступно. Его собирают из двух запросов: LEFT JOIN плюс RIGHT JOIN с условием IS NULL по ключу левой таблицы, объединёнными оператором UNION.

СХЕМА ИМИТАЦИИ

```
SELECT P.ProductName, C.CategoryName
FROM Products AS P LEFT JOIN Categories AS C ON P.CategoryID = C.ID
UNION
SELECT P.ProductName, C.CategoryName
FROM Products AS P RIGHT JOIN Categories AS C ON P.CategoryID = C.ID
WHERE P.CategoryID IS NULL;
```



A ∪ B — всё из обеих таблиц



= FULL OUTER JOIN
все записи обеих таблиц

UNION убирает повторы: общая часть попадает в оба запроса, но в итоге останется ровно один раз

6 Преобразование типов и вычисления

Nz · IsNull · Val · CCur · CInt / CLng · CDate · Replace · IIf — схема «исходное значение → функция → результат»

В русской локали Access аргументы разделяются ;

Nz замена Null значением

Возвращает поле, а если оно Null — подставляет запасное значение. Спасает суммы и конкатенации.

СИНТАКСИС И ПРИМЕР

`Nz([UnitPrice]; 0)`



Результат: `Nz([UnitPrice];0)*1.2` посчитается даже там, где цена не заполнена.

Val извлечение числа из текста

Читает число из начала текстовой строки; запятую заранее меняют на точку через Replace.

СИНТАКСИС И ПРИМЕР

`Val(Replace([UnitPrice]; ";", "."))`



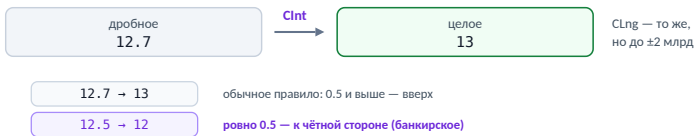
Результат: текстовая цена стала числом — с ней работают сравнения и Sum/Avg.

CInt · CLng целые числа

CInt — целое в диапазоне Integer, CLng — длинное целое для больших значений и ключей.

СИНТАКСИС И ПРИМЕР

`CInt([Quantity])` `CLng([ID])`



Результат: `12.7 → 13` (округление к чётному банкирское: `12.5 → 12`).

Replace замена символов

Заменяет в строке все вхождения одной подстроки другой — чистка дефисов, разделителей, кавычек.

СИНТАКСИС И ПРИМЕР

`Replace([ProductName]; "-"; " ")`



Результат: каждое вхождение «-» заменено пробелом; поиска и замены по таблице это не меняет.

IsNull проверка на Null

Возвращает Да/True, если поле пустое. В отличие от `= Null` работает всегда.

СИНТАКСИС И ПРИМЕР

`IsNull([CategoryGroup])`



Результат: `WHERE IsNull([CategoryGroup])` — то же, что `WHERE CategoryGroup IS NULL`.

CCur приведение к денежному типу

Превращает число или текст в денежное значение — убирает ошибки округления при расчётах.

СИНТАКСИС И ПРИМЕР

`CCur(Val([UnitPrice]))`



Результат: значение готово для денежного поля и точных сумм в отчётах.

CDate приведение к дате

Превращает текст или число в настоящую дату — после этого работают Year, DateAdd, DateDiff.

СИНТАКСИС И ПРИМЕР

`CDate("01.03.2024")`



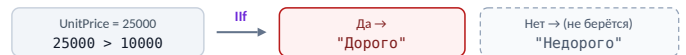
Результат: строка стала датой; в SQL литералы дат записываются между #.

IIf условное значение

Мгновенное «если... то... иначе» прямо в выражении: условие; значение если Да; значение если Нет.

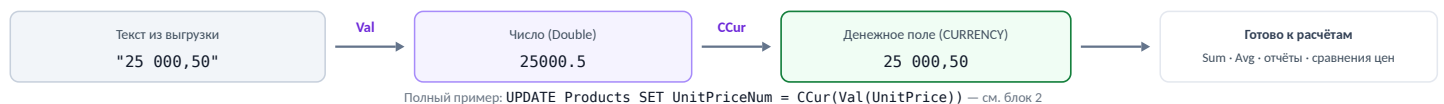
СИНТАКСИС И ПРИМЕР

`IIf([UnitPrice] > 10000; "Дорого"; "Недорого")`



Результат: для цены 25000 выражение вернёт `Дорого`; для 1200 — `Недорого`.

Цепочка очистки денежного значения — текст → Val → CCur → денежное поле



Каждый шаг превращает «грязное» значение из выгрузки в тип, с которым Access умеет работать: сначала число, затем деньги.

& объединение строк

Склеивает текст, числа и поля в одну строку; пробелы между частями нужно ставить самому.

СИНТАКСИС И ПРИМЕР

```
[Brand] & " " & [ProductName]
```



Результат: "Logitech Мышь MX". Если хоть одно поле Null — вся строка станет Null: оберните поля в Nz (блок 6).

Right последние символы

Отрезает символы справа — год из кода документа, расширение файла, суффикс.

СИНТАКСИС И ПРИМЕР

```
Right([КодДокумента]; 4)
```



Результат: "2024" — год выделяется из кода без разбора строки вручную.

Len длина строки

Считает символы в строке — контроль заполнения полей перед загрузкой в поле фиксированного размера.

СИНТАКСИС И ПРИМЕР

```
Len([ProductName])
```



Результат: "Стол" → 4; пробелы внутри тоже считаются.

Trim пробелы по краям

Срезает начальные и конечные пробелы; внутренние пробелы не трогает.

СИНТАКСИС И ПРИМЕР

```
Trim([ProductName])
```



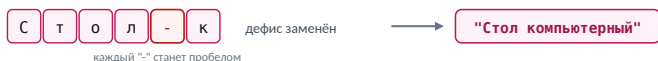
Результат: "Стол" → "Стол" — чистые значения для сравнений и группировок.

Replace замена в строке

Меняет все вхождения подстроки — нормализация разделителей и дефисов перед загрузкой данных.

СИНТАКСИС И ПРИМЕР

```
Replace([ProductName]; "-", " ")
```



Результат: "Стол-компьютерный" → "Стол компьютерный"; схема преобразования типов — в блоке 6.

Left первые символы

Отрезает заданное количество символов слева — коды, префиксы артикулов.

СИНТАКСИС И ПРИМЕР

```
Left([ProductName]; 5)
```



Результат: "Комп" из строки "Компьютерный стол".

Mid подстрока с позиции

Берёт символы начиная с указанной позиции (нумерация с 1) и в заданном количестве.

СИНТАКСИС И ПРИМЕР

```
Mid([ProductName]; 2; 3)
```



Результат: "тол" из строки "Стол" — позиции 2, 3, 4.

UCase · LCase регистр

UCase поднимает строку в верхний регистр, LCase опускает в нижний — унификация справочников.

СИНТАКСИС И ПРИМЕР

```
UCase([Category]) LCase([Category])
```



Результат: "Авто" → "ABTO" или "авто" — единый регистр для GROUP BY.

InStr позиция подстроки

Возвращает номер позиции, с которой начинается подстрока; 0 — если не найдена.

СИНТАКСИС И ПРИМЕР

```
InStr([ProductName]; "телефон")
```



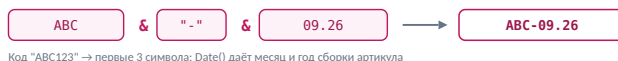
Результат: "Стол телефонный" → 6 — подстрока начинается с 6-й позиции.

Практика: артикул из частей

Строковые функции сочетаются в одном выражении: код, разделитель и метка даты собираются в артикул.

СИНТАКСИС И ПРИМЕР

```
Left([Код]; 3) & "-" & Format(Date(); "mm.yy")
```



Результат: "ABC-09.26" — артикул собирается заново при каждом запуске запроса.

Date · Now

сегодня; сегодня и время

Date() возвращает текущую дату без времени, Now() — дату со временем. Без аргументов.

СИНТАКСИС И ПРИМЕР

Date() Now()



Результат: значения вычисляются в момент запуска запроса — удобно для «просрочено ли?».

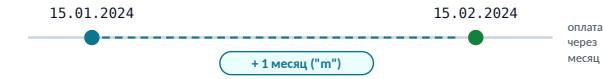
DateAdd

прибавить интервал

Сдвигает дату на заданное число интервалов: "yyyy" — годы, "m" — месяцы, "d" — дни.

СИНТАКСИС И ПРИМЕР

DateAdd("m"; 1; [OrderDate])



Результат: 15.01.2024 → 15.02.2024 — срок оплаты, гарантия, следующая поставка.

Format

дата в текст по шаблону

Превращает дату в строку нужного вида — для выгрузок и подписей в отчётах.

СИНТАКСИС И ПРИМЕР

Format([OrderDate]; "dd.mm.yyyy")



Результат: строка "15.01.2024" ; шаблон "мм.уууу" даёт метку периода "01.2024" .

Year · Month · Day

части даты

Выделяют из даты год, номер месяца и день — для группировки по периодам.

СИНТАКСИС И ПРИМЕР ([ORDERDATE] = 15.01.2024)

Year([OrderDate]) Month([OrderDate]) Day([OrderDate])



Результат: из одной даты получаются три числа — строительные блоки для отчётов по периодам.

DateDiff

разница между датами

Считает, сколько интервалов помещается между двумя датами: дней, месяцев, лет.

СИНТАКСИС И ПРИМЕР

DateDiff("d"; [OrderDate]; Date())



Результат: 47 — возраст заказа в днях; с "m" и "yyyy" — возраст в месяцах и годах.

Даты в условиях WHERE

Литералы дат заключаются между решётками; диапазон — через BETWEEN или пару неравенств.

ПРИМЕРЫ

WHERE OrderDate = #2024-01-15#
WHERE OrderDate BETWEEN #2024-01-01#
AND #2024-01-31#

OrderDate в январе?

#2024-01-15# ✓
#2024-02-03# ✗ вне диапазона



Результат: отбор заказов за день или за период; год и месяц удобно брать функциями Year/Month.

Мини-календарь — как функции читают дату

Один и тот же лист календаря отвечает на три вопроса: какой год, какой месяц, какой день.



Результат: GROUP BY Year(...), Month(...) режет таблицу на периоды, Day() выделяет день внутри месяца.

Практика: возраст и просрочка

DateDiff и If вместе превращают даты в бизнес-статусы прямо в запросе.

ПРИМЕР — ФЛАГ ПРОСРОЧКИ

If(DateDiff("d"; [OrderDate]; Date()) > 30;
"Просрочен"; "В работе")

OrderDate	статус
возраст 47 дней	Просрочен
возраст 5 дней	В работе

Результат: каждому заказу присвоен статус по возрасту; порог 30 дней меняется одним числом.

Count количество строк

Count(*) считает все строки выборки; Count(поле) — только заполненные (без Null).

СИНТАКСИС И ПРИМЕР

```
SELECT Count(*) AS Количество
FROM Products;
```

ProductName	Стол
Стул	
Лампа	
Мышь	

→ Count

Количество	4
------------	---

Count = 4

все строки → одно число

Count(поле) не считает Null-строки

Результат: одно число — сколько записей удовлетворило запросу.

Sum · Avg сумма и среднее

Sum складывает значения столбца, Avg считает среднее. Работают только с числами и деньгами.

СИНТАКСИС И ПРИМЕР

```
SELECT Sum(UnitPrice) AS Сумма,
       Avg(UnitPrice) AS СредняяЦена
FROM Products;
```

25000

4500

1200

Avg = 10 233

Sum = 30 700

цены товаров: среднее — уровень, сумма — весь столбик

Результат: Сумма 30700 и СредняяЦена 10233,33 — итог по всей таблице разом.

Min · Max минимум и максимум

Крайние значения столбца: самая дешёвая и дорогая позиция, первая и последняя дата.

СИНТАКСИС И ПРИМЕР

```
SELECT Min(UnitPrice) AS Минимум,
       Max(UnitPrice) AS Максимум
FROM Products;
```

Мышь · 800

Лампа · 1200

Стул · 4500

Стол · 25000

Min = 800

работают и с датами:

Min = первая, Max = последняя

Max = 25000

Результат: вилка цен 800 ... 25000 — сразу видно разброс ассортимента.

GROUP BY группировка

Делит строки на группы по значению поля и считает агрегат для каждой группы отдельно.

СИНТАКСИС И ПРИМЕР

```
SELECT Category, Count(*) AS Количество
FROM Products
GROUP BY Category;
```

Category	товар
Электроника	Мышь
Электроника	Лампа
Авто	Щётка

→ группы

Category	Кол-во
Электроника	2
Авто	1

Результат: каждая категория — одна строка итога; всё, что вне SELECT-агрегатов, попадает в GROUP BY.

HAVING фильтр по группам

WHERE отбирает строки до группировки, HAVING — уже готовые группы по значению агрегата.

СИНТАКСИС И ПРИМЕР

```
SELECT Category, Avg(UnitPrice) AS СредняяЦена
FROM Products
GROUP BY Category
HAVING Avg(UnitPrice) > 10000;
```

группа	Avg проходит?
Электроника	18500 ✓ > 10000
Мебель	4500 ✗ отсеяна

18500

4500

порог HAVING > 10000

столбики ниже пунктира отсеиваются целиком

Результат: остались только «дорогие» категории; агрегат в HAVING можно писать повторно.

Мини-диаграммы итогов по категориям

То, что считают Sum, Count и Avg по группам — на глаз, bar-диаграммой с подписями значений.

Count — сколько товаров в категории

2

1

2

Эл-ка

Авто

Мебель

Sum — выручка категории

2,0

85,0

2,0 тыс

85,0 тыс

Avg — средняя цена

18,5

85,0

14,8

18,5 тыс

85,0 тыс

14,8 тыс

штрих-сетка не нужна: значения подписаны

Результат: диаграммы строятся из того же запроса с GROUP BY — Count, Sum, Avg по каждой категории.

Переход: таблица записей → группировка → итоговая таблица

Products — записи

Электроника · Мышь · 800

Электроника · Лампа · 1200

Авто · Щётка · 350

Мебель · Стол · 25000

Мебель · Стул · 4500

→ GROUP BY

строки собраны в группы

Электроника 2 шт · 2 000

Авто 1 шт · 350

Мебель 2 шт · 29 500

→ Count · Sum

итоговая таблица

Category	Count (*)	Sum
Электроника	2	2000
Авто	1	350
Мебель	2	29500

SELECT Category, Count(*) AS Количество, Sum(UnitPrice) AS Сумма FROM Products GROUP BY Category

Результат: пять товарных строк превратились в три итоговые — по одной на каждую группу; агрегаты считаются внутри групп.

Microsoft Access — SQL и функции · расширенный набор карточек · блок 9 из 12

Access 2007–2016 · агрегаты

Свойства поля в Конструкторе таблиц

Для поля Category задаётся тип «Короткий текст», ограничение длины и список разрешённых значений через условие In (...); значения внутри условия разделяются точкой с запятой.

свойство	значение
Тип данных	Короткий текст
Размер поля	40
Условие на значение	In ("Дом и мебель"; "Красота и здоровье"; "Электроника"; "Авто")
Сообщение об ошибке	"Выберите категорию из списка"

Результат: Access проверяет каждое введённое значение ещё до сохранения записи — мусор в справочник не попадёт.

Как проходит проверка

Значение из списка — запись сохраняется; любое иное, включая «Прочее», Access отклонит и покажет сообщение.

вводимое значение	проверка In (...)
"Дом и мебель"	✓ принято
"Красота и здоровье"	✓ принято
"Электроника"	✓ принято
"Авто"	✓ принято
"Прочее"	✗ отклонено

Microsoft Access

Выберите категорию из списка

шлюз In (...)

● ● ● ● ●

4 значения из списка — проходят насквозь «Прочее» отскакивает

Результат: «Прочее» не входит в список In (...) — Access не даст сохранить запись, пока значение не исправят.

Проверка и SQL-ввод

Условие на значение действует и на INSERT — «запрещённую» строку запрос вставить не сможет.

ПРИМЕР — ДОПУСТИМОЕ ЗНАЧЕНИЕ

```
INSERT INTO Products (ProductName, Category)
VALUES ("Мышь", "Электроника");
```

ПРИМЕР — ОТКЛОНЯЕМОЕ ЗНАЧЕНИЕ

```
INSERT INTO Products (ProductName, Category)
VALUES ("Чехол", "Прочее");
```

запрос	итог
VALUES ("Мышь", "Электроника")	✓ добавлено
VALUES ("Чехол", "Прочее")	✗ нарушение

VALUES ("Мышь", "Электроника")

условие на In (...)

✓ запись добавлена

VALUES ("Чехол", "Прочее")

условие на In (...)

✗ Access отклонил

Зачем нужна проверка

Единый список категорий делает группировки и фильтры предсказуемыми: «Электроника» и «прочее» не размыивают отчёты десятками вариантов.

справочник категорий

Дом и мебель
Красота и здоровье
Электроника · Авто

In (...)

Products.Category

только значения из списка – иначе ошибка ввода

Результат: GROUP BY Category даёт ровно четыре группы — отчёты сходятся со справочником.

Размер поля и длинные строки

Размер 40 ограничивает длину «Короткого текста»: строка длиннее 40 символов не сохранится.

ПРОВЕРКА ДЛИНЫ ВЫРАЖЕНИЕМ

```
WHERE Len([Category]) > 40
```

значение	длина / итог
"Красота и здоровье"	19 • ✓ влезает
41 символ и длиннее	✗ превышение

"Красота и здоровье" · 19

19 ≤ 40 — влезает; 41-й символ уже не соо

лимит 40

Результат: перед загрузкой данных длину проверяют функцией Len (блок 7), тип поля — блоком 11.

11 Типы данных Access

Короткий текст · Длинный текст · Числовой · Денежный · Дата и время · Счётчик · Да/Нет · Вложение

Название в Конструкторе ↔ SQL-тип в CREATE TABLE

Короткий текст

До 255 символов: названия, артикулы, категории. Размер задаётся свойством поля.

ProductName TEXT(100)
-- пример значения: "Стол"

Стол

4 символа из 40 доступных

С

Т

О

Л

каждый символ на своём месте; лишние ячейки остаются пустыми

Пример поля: ProductName TEXT(100) — см. блок 10 про размер и проверку.

Числовой

Размеры поля: LONG (целое), DOUBLE (дробное), INTEGER и другие — под задачу.

Quantity LONG
UnitPriceNum DOUBLE
-- пример значения: 25

Quantity LONG · 25

UnitPriceNum DOUBLE · 25,5

Reserve INTEGER · -32768-32767

размер поля выбирается под диапазон значений — лишний объём не нужен

Пример поля: Quantity LONG ; ключевые связи с AUTOINCREMENT — только LONG (блоки 1 и 5).

Дата и время

DATETIME хранит дату и время разом; литералы в SQL — между решётками #...#.

OrderDate DATETIME
-- пример: #2024-03-01#

МАРТ

01

#03-01-2024#

#03-01-2024 14:30#

дата и время хранятся разом — сортировка хронологии работает сама; без времени — ровно полночь

Пример поля: OrderDate DATETIME ; функции дат — блок 8.

Да/Нет

Логический флажок: Да (True, -1) или Нет (False, 0) — активность, оплата, отметки.

IsActive YESNO
-- пример: WHERE IsActive = True

True · Да · -1

False · Нет · 0

WHERE IsActive = True отбирает только активные

Пример поля: IsActive YESNO ; в WHERE пишут = True или = False .

Длинный текст

Текст без ограничения длины (Мемо): описания, комментарии, примечания.

Description LONGTEXT
-- пример: описание товара
-- на несколько абзацев

Описание: мощный компьютер для работы с графикой и большими таблицами. Подходит для видеомонтажа и CAD. текст продолжается без лимита длины.

полоса прокрутки в форме, несколько абзацев; в GROUP BY почти не участвует

Пример поля: Description LONGTEXT ; в группировках и JOIN участвует редко.

Денежный

CURRENCY — фиксированные 4 знака после запятой: точные суммы без ошибок округления.

UnitPrice CURRENCY
-- пример значения: 25 000,50

25 000,50

4 знака после запятой — суммы сходятся копейка в копейку, ошибок округления нет даже на миллионных значениях

1 250 000,00

Пример поля: UnitPrice CURRENCY ; текст в деньги превращает CCur (блок 6).

Счётчик

AUTOINCREMENT: Access сам присваивает 1, 2, 3... — идеальный первичный ключ.

ID AUTOINCREMENT PRIMARY KEY
-- значения: 1, 2, 3, ...

1

2

3

4

5...

значения не повторяются — идеальный ключ
номер присваивает сам Access; удалённые номера не переиспользуются

Пример поля: ID AUTOINCREMENT PRIMARY KEY ; на него ссылается FK типа LONG.

Вложение

Файлы (картинки, PDF) внутри записи; через SQL DDL не создаётся — только в Конструкторе.

-- в CREATE TABLE недоступен,
-- добавляется в Конструкторе
-- пример: скан счёта в записи

счёт_124.pdf

фото_товара.jpg

договор.docx

несколько файлов в одной записи

Пример: поле СканСчёта с вложенным PDF; в запросах участвует как ссылка на файл.

Как выбрать тип: короткая шпаргалка выбора

Деньги → CURRENCY, не DOUBLE

Ключи → AUTOINCREMENT + LONG

Текст до 255 симв. → TEXT(n)

Момент времени → DATETIME

Результат: правильный тип с самого начала избавляет от пересоздания полей (ALTER COLUMN, блок 1) и ошибок округления.

Окно «Введите значение параметра»

При запуске запроса Access неожиданно просит ввести значение — хотя вы ничего не спрашивали.

причина

Опечатка в имени поля: **UnitPrise** вместо UnitPrice — Access считает это неизвестным параметром.

Исправление: сверить имена полей с Конструктором таблицы; поле есть, но не в этом запросе? Добавьте таблицу.

Введите значение параметра

UnitPrice

Отмена

ОК

симптом, а не вопрос

Access не нашёл поле «UnitPrise» и решил, что это параметр запроса

Правильно: UnitPrice — сверьте имя в Конструкторе таблицы

«Ошибка синтаксиса (пропущен оператор)»

Запрос не запускается, Access ругается на пропущенный оператор или разделитель.

причина

Неправильный разделитель аргументов или кавычки: `IIf([Цена] > 10000, "Да", "Нет")` — запятые в русской локали.

Исправление: в выражениях — точка с запятой; текст — в двойных кавычках: `IIf([Цена] > 10000; "Да"; "Нет")`

ошибка ок

<code>; → , в выражении</code>	✗
<code>'Одинарные' кавычки</code>	✗
<code>IIf (а; "Да"; "Нет")</code>	✓

Ошибка синтаксиса (пропущен оператор)

`IIf ([Цена] > 10000, "Да", "Нет")`

↑ запятые вместо ; — Access спотыкается

две частые причины

запятые в выражениях (русская локаль)

'одинарные' кавычки вместо "двойных"

Null и пустая строка «»

Поле «пустое» по-разному: Null — значение отсутствует вообще, "" — введена пустая строка.

причина

Условие `= ""` не находит поля с Null, а `IS NULL` не находит пустые строки — часть записей «пропадает».

Исправление: искать оба случая сразу: `WHERE Поле IS NULL OR Поле = ""` — или чистить данные `Trim`’ом.

значение поля `IS NULL = ""`

Null	✓	✗
"" (пустая строка)	✗	✓

Null — пусто совсем

IS NULL видит только верхнюю, = "" — только нижнюю; ловите оба

"" — введена пустая строка

WHERE Поле IS NULL OR Поле = ""

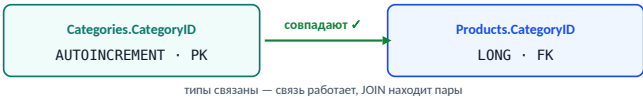
Неправильный тип PK / FK

Связь не создаётся или JOIN не находит пар: типы связанных полей различаются.

причина

Ключ AUTOINCREMENT связан с текстом или числом не того размера — сравнение значений ломается.

Исправление: с полем AUTOINCREMENT связывать поле типа LONG — и никак иначе.



Даты без решёток #

Дата в условии записана как текст — сравнение не срабатывает или выдаёт мусор.

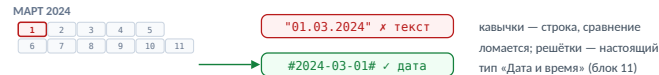
причина

`WHERE OrderDate = "01.03.2024"` — кавычки превращают дату в строку.

Исправление: литералы дат между решётками: `WHERE OrderDate = #2024-03-01#`

запись условия итог

<code>= "01.03.2024"</code>	✗ текст
<code>= #2024-03-01#</code>	✓ дата



UPDATE / DELETE без проверки

Команда выполнена «не на тех» строках — а отменить изменение в Access нельзя.

причина

Слишком широкое WHERE — или его забыли совсем: `DELETE FROM Products`; очистит всю таблицу.

Исправление: сначала SELECT с тем же WHERE, убедиться в списке строк — и только потом менять/удалять.



Чек-лист перед запуском запроса

- Имена полей сверены ✓
- Текст в "кавычках" ✓
- Выражения через ; ✓
- Даты между ## ✓

Результат: четыре быстрые проверки отсекают большинство ошибок до запуска; подробности — в блоках 4, 6 и 8.